Healthcare Patient Management System — System Design Document

Architectural Decisions, Relational Modeling, Security Design, and Trade-offs

1. Executive Summary & Problem Statement

Modern health systems manage massive influxes of inpatient, demographic, pharmacological, and diagnostic records across disparate facilities. Inefficiencies in manual or uncurated handling lead to severe administrative bottlenecks, obscured readmission signals, and delayed clinical interventions. This system implements an enterprise-grade full-stack architecture that transforms over 100,000 historical hospital records (UCI Diabetes 130-US Hospitals dataset) into structured 3NF relational schemas, exposes secure RESTful CRUD and analytics microservices, and serves responsive clinical dashboards.

2. Architectural Design & Layering

The application is engineered as a decoupled, multi-tier service:
• **Data Engineering Layer (ETL):** Automated ingestion, deduplication, missing-data imputation, and CDC incremental loading.
• **Data Storage Layer:** Adaptive relational core supporting MySQL for enterprise production and self-initializing SQLite for zero-friction local evaluations.
• **API Microservice Layer:** FastAPI backend enforcing Argon2 password hashing, OAuth2 Bearer JWT authorization, and strict Role-Based Access Control (RBAC).
• **User Interface Layer:** Streamlit clinical portal designed with a corporate healthcare visual identity (Navy, Teal, Slate).

3. Relational Schema Normalization (3NF)

The raw flat dataset (50 attributes) is decomposed into 7 normalized relational tables to eliminate data redundancy and preserve referential integrity:
• **patients:** Master demographic registry (patient_nbr [PK], race, gender).
• **encounters:** Transactional inpatient facts (encounter_id [PK], patient_nbr [FK], admission_type_id [FK], time_in_hospital, readmitted, etc.).
• **admission_types, discharge_dispositions, admission_sources:** Terminology lookup tables.
• **encounter_diagnoses:** Composite primary key (encounter_id, diagnosis_position) storing ICD-9 diagnostic codes.
• **encounter_medications:** Composite primary key (encounter_id, medication) mapping active pharmacological treatments.
• **audit_logs:** Immutable compliance ledger capturing user_id, action, resource, outcome, and timestamp.

4. Security Architecture & Role-Based Access Control (RBAC)

Role Name	Permitted Operations	Access Scope
Admin	User provisioning, schema maintenance, record deletion, audit inspection	Unrestricted system-wide
Healthcare Staff	Patient registration, encounter creation/updating, clinical history lookup	Clinical operations & encounters
Analyst	Aggregate reporting, KPI trend analysis, readmission insights, CSV/Excel export	Read-only analytics & reporting

5. Design Decisions & Trade-Offs

• **Dual-Engine Database Strategy:** Production MySQL provides multi-concurrent enterprise scaling. An automatic fallback to local SQLite eliminates local installation friction for academic evaluation without compromising foreign keys.

• **FastAPI vs Flask:** FastAPI was chosen for native asynchronous high-throughput concurrency, automatic OpenAPI/Swagger schema documentation, and Pydantic v2 validation.

• **Argon2 Password Hashing:** Chosen over legacy bcrypt to provide memory-hard, GPU-resistant credential storage adhering to OWASP recommendations.